

Projeto empréstimo de livros.

Vamos criar um banco de dados empréstimo

```
create database bdEmprestimo;
use bdEmprestimo;

create table tbUsuario(
codUsu int primary key auto_increment,
nomeUsu varchar(50)
);

create table tbLivro(
codlivro int primary key auto_increment,
nomelivro varchar(50),
imagemLivro varchar(255)
);

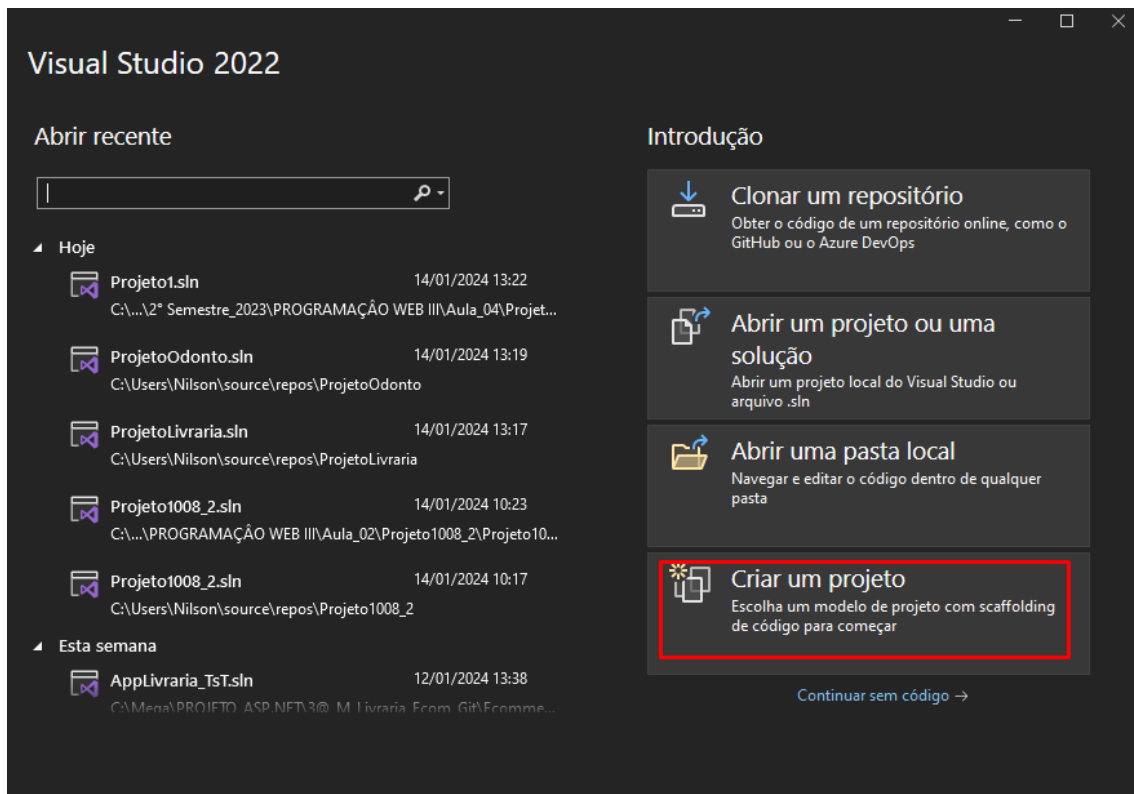
create table tbEmprestimo(
codEmp int primary key auto_increment,
dataEmp varchar(20),
dataDev varchar(20),
codUsu int references tbUsuario(codUsu)
);

create table itensEmp(
codItem int primary key auto_increment,
codEmp int references tbEmprestimo(codEmp),
codLivro int references tbLivro(codLivro)
)

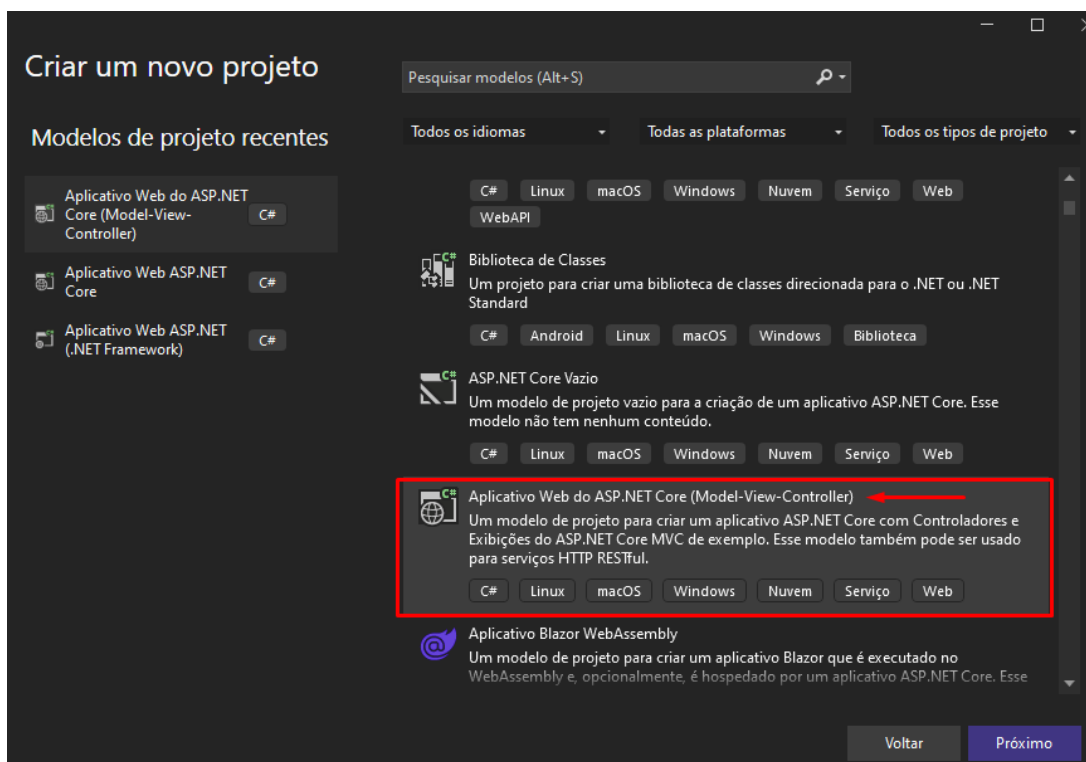
select * from itensEmp;

insert into tbUsuario values (default,'Nilson'),(default,'Bruno')
```

Vamos criar o projeto no Visual Studio 2022



## Aplicação Web do ASP.NET Core (Model-View-Controller)



Vamos colocar o nome ProjetoEmprestimoAspCore

### Configurar seu novo projeto

Aplicativo Web do ASP.NET Core (Model-View-Controller) C# Linux macOS Windows Nuvem  
Serviço Web

Nome do projeto  
ProjetoEmprestimoAspCore

Local  
C:\Users\Nilson\source\repos\ASPNETCORE ...

Nome da solução ⓘ  
ProjetoEmprestimoAspCore

☐ Colocar a solução e o projeto no mesmo diretório

Deixar a opção configurar para HTTPS desmarcada depois clica em Criar

### Informações adicionais

Aplicativo Web do ASP.NET Core (Model-View-Controller) C# Linux macOS Windows Nuvem  
Serviço Web

Estrutura ⓘ  
.NET 7.0 (Suporte a Termos Padrão)

Tipo de autenticação ⓘ  
Nenhum

☐ Configurar para HTTPS ⓘ

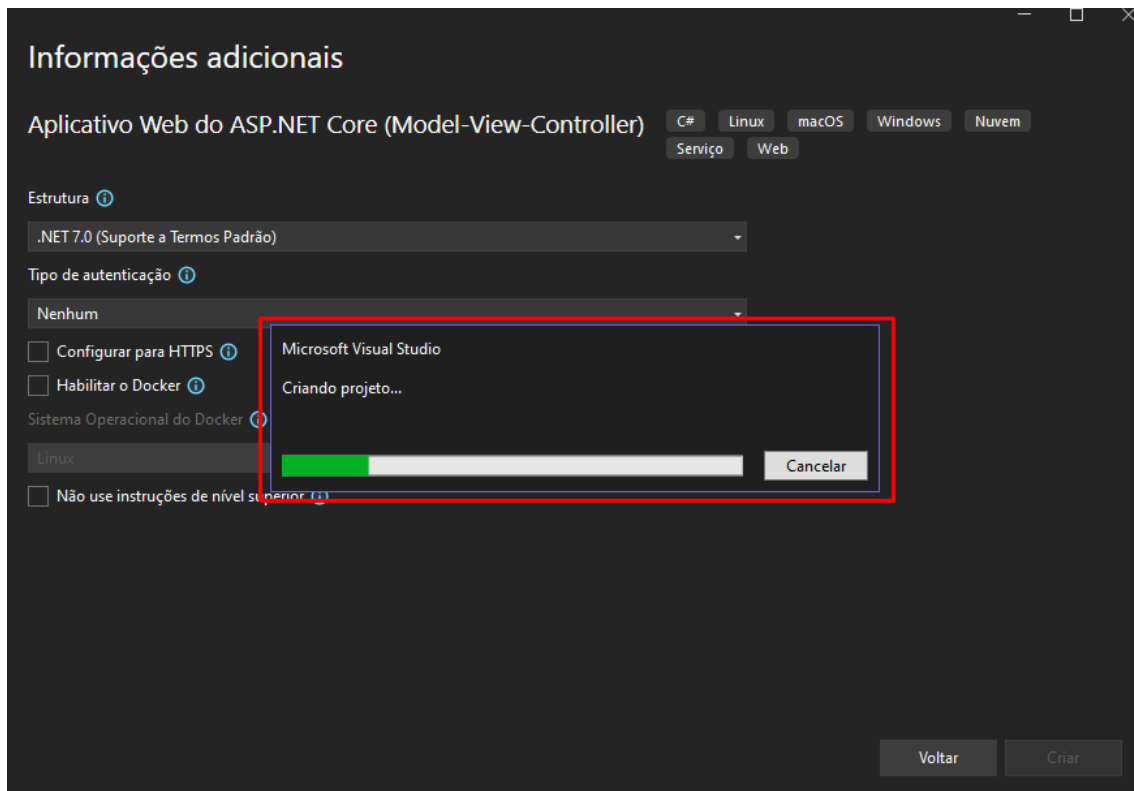
☐ Habilitar o Docker ⓘ

Sistema Operacional do Docker ⓘ  
Linux

☐ Não use instruções de nível superior ⓘ

Voltar Criar

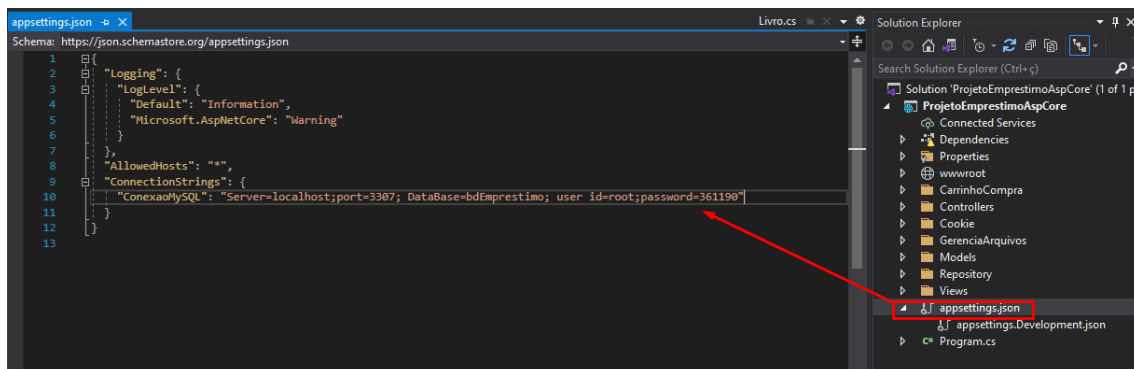
O processo de criação do projeto iniciará, vamos aguardar ....



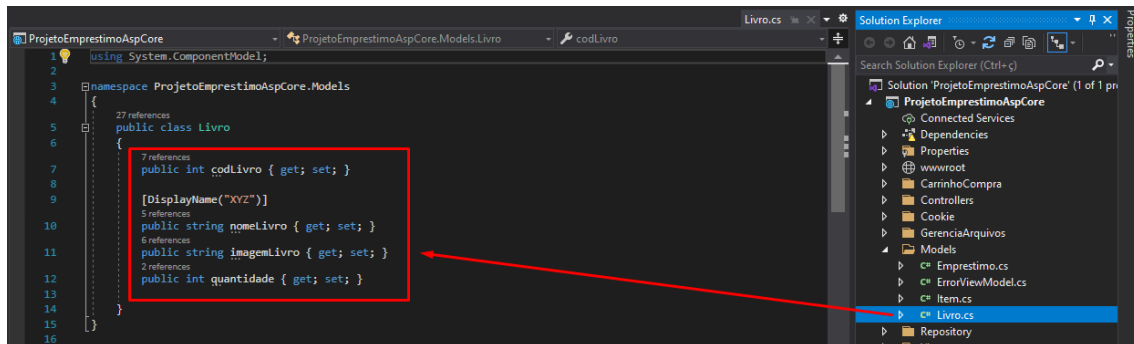
Pronto. Após o projeto ser criado vamos aguardar carregar todas as dependências.

Vamos executar o nosso projeto, sem ter add nenhum arquivo, para testar se está funcionando. \* **Dica** \* – Sempre faça este procedimento.

Vamos adicionar a String de conexão com MySql



Adicione uma classe na pasta **Model** com o nome **Livro**



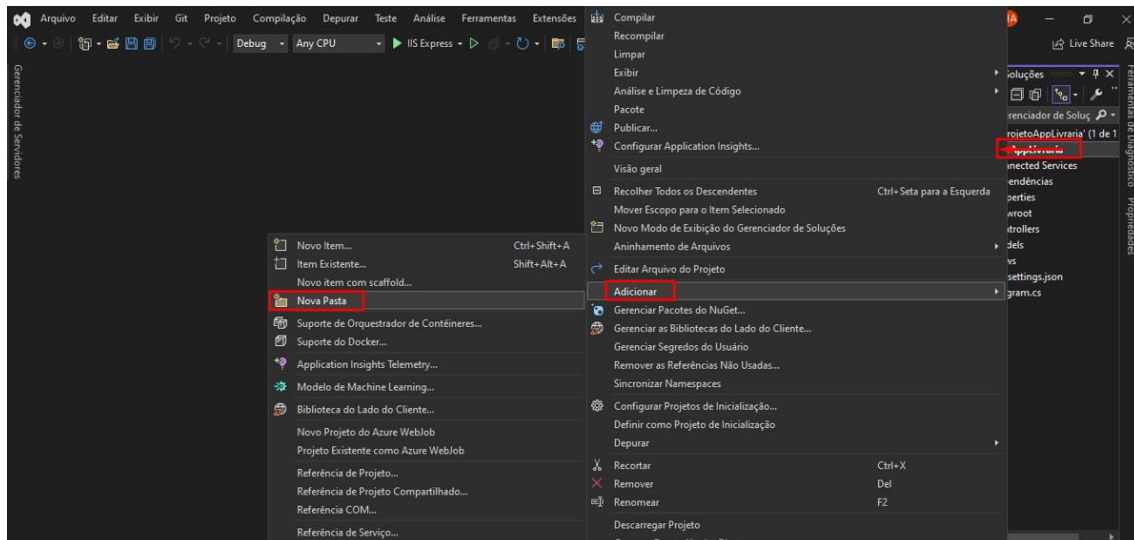
Adicione uma classe na pasta **Model** com o nome **Itens**



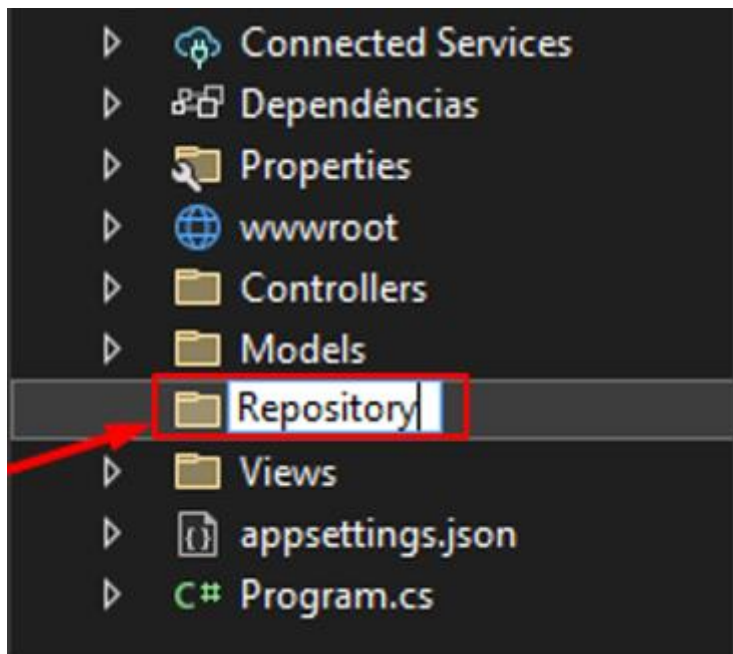
Adicione uma classe na pasta **Model** com o nome **Emprestimo**



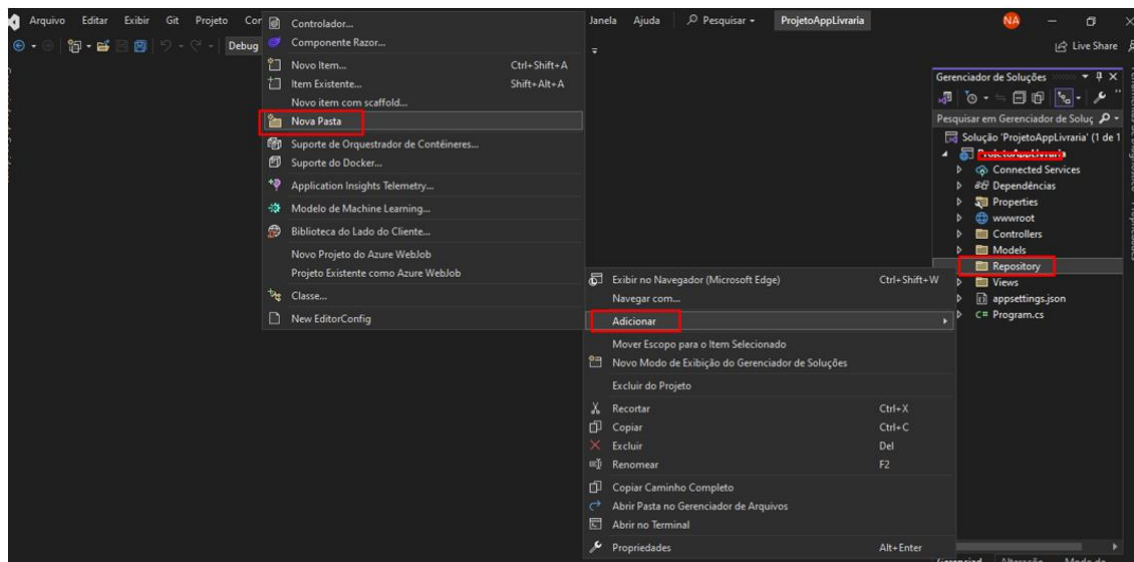
Clica com botão direito sobre o projeto – Adicionar – nova pasta



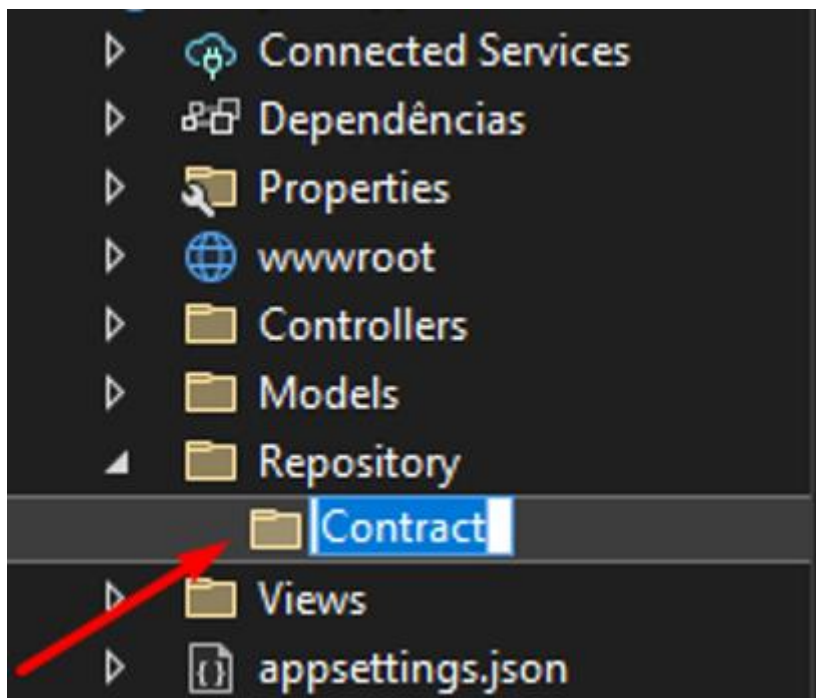
Vamos renomear para **repository**



Clicando com o botão direito sobre a pasta Repository

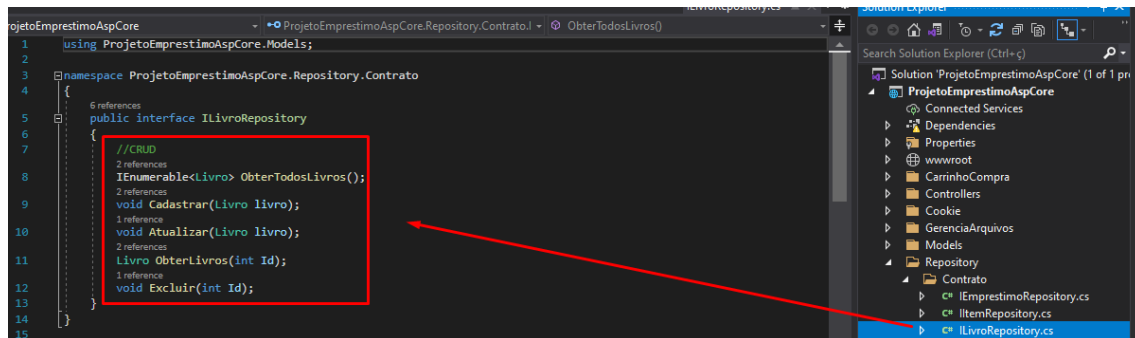


Vamos colocar o nome de **Contract**



Agora vamos adicionar um Interface chamada `ILivroRepository`. Que é uma classe abstrata, nesta classe ficará somente a assinatura dos métodos.

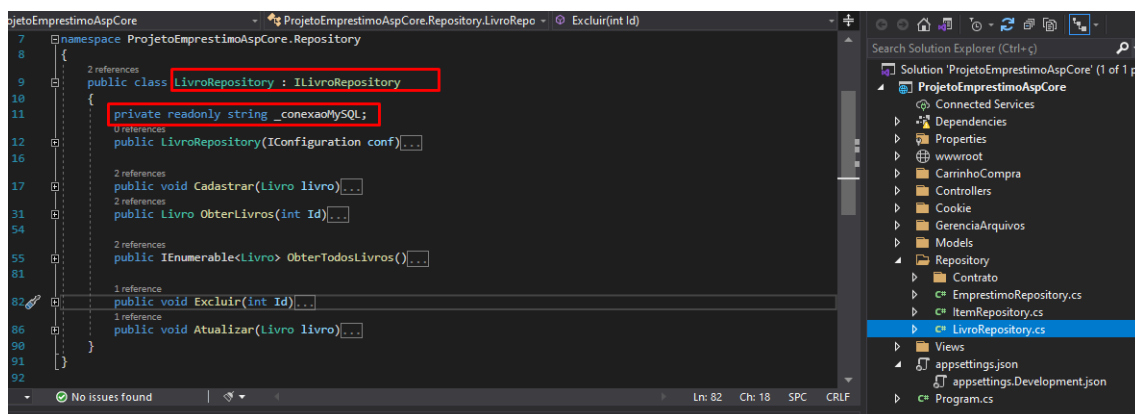
- Clicando com o botão direito sobre a pasta Contract, depois em adicionar – Classe



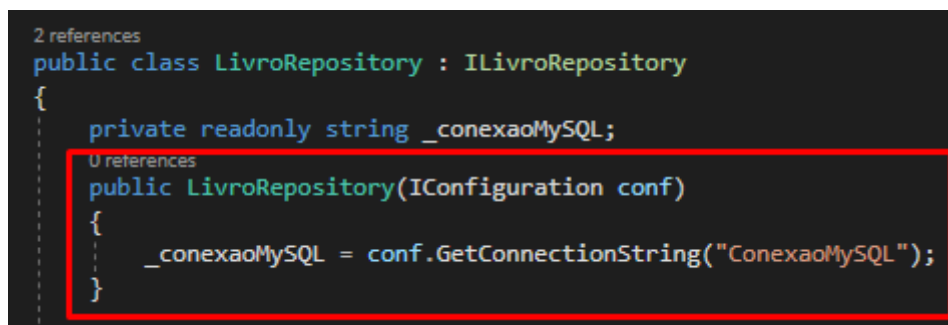
Agora vamos adicionar uma classe concreta (**LivroRepository**), que implementa os métodos da nossa classe abstrata.

Após criar a classe vamos colocar um Extends para interface

Depois colocar a conexão com o banco de dados, passando uma propriedade privada.



Vamos usar a conexão como parâmetro no construtor da classe.



Vamos implementar os métodos da interface e depois reescrever os métodos



```

2 references
public class LivroRepository : ILivroRepository
{
    private readonly string _conexaoMySQL;
    0 references
    public LivroRepository(IConfiguration conf) {...}

    2 references
    public void Cadastrar(Livro livro) {...}
    2 references
    public Livro ObterLivros(int Id) {...}

    2 references
    public IEnumerable<Livro> ObterTodosLivros() {...}

    1 reference
    public void Excluir(int Id) {...}
    1 reference
    public void Atualizar(Livro livro) {...}
}

```

Vamos criar um método para cadastrar o livro no **LivroRepository**

```

4 references
public void Cadastrar(Livro livro)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();

        MySqlCommand cmd = new MySqlCommand("insert into tbLivro values(default, @NomeLivro, @ImagemLivro)", conexao);

        cmd.Parameters.Add("@NomeLivro", MySqlDbType.VarChar).Value = livro.nomeLivro;
        cmd.Parameters.Add("@ImagemLivro", MySqlDbType.VarChar).Value = livro.imagemLivro;
        cmd.ExecuteNonQuery();
        conexao.Close();
    }
}

```



Vamos criar um método para obter os livros por ID na classe **LivroRepository**

```

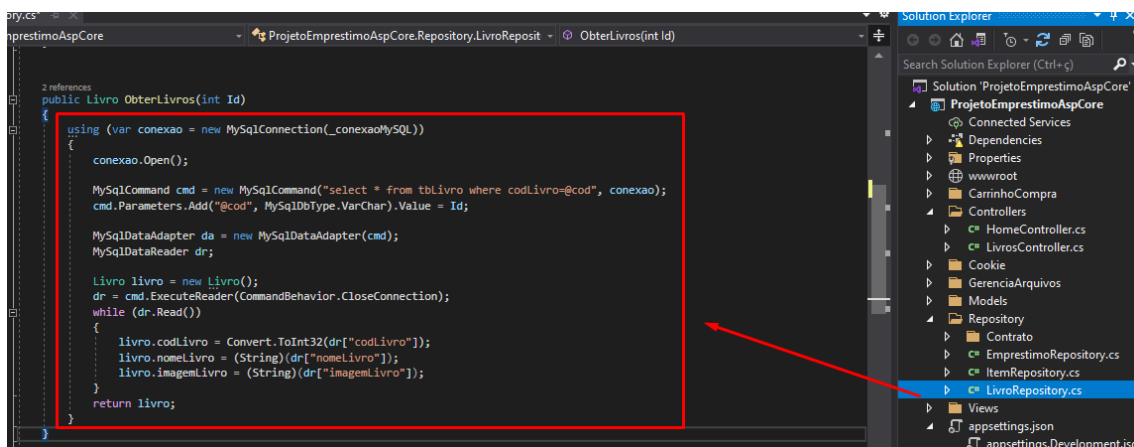
2 references
public Livro ObterLivros(int Id)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();

        MySqlCommand cmd = new MySqlCommand("select * from tbLivro where codLivro=@cod", conexao);
        cmd.Parameters.Add("@cod", MySqlDbType.VarChar).Value = Id;

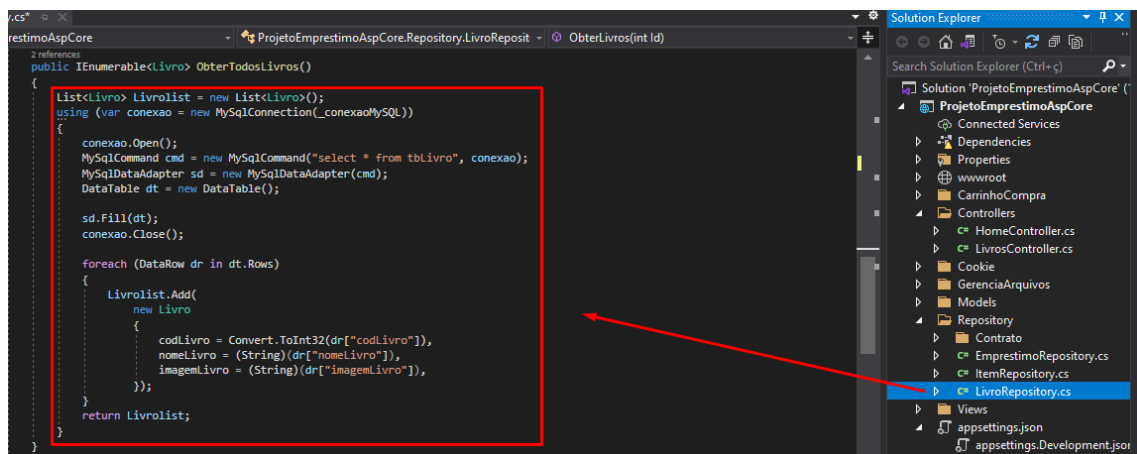
        MySqlDataAdapter da = new MySqlDataAdapter(cmd);
        MySqlDataReader dr;

        Livro livro = new Livro();
        dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
        while (dr.Read())
        {
            livro.codLivro = Convert.ToInt32(dr["codLivro"]);
            livro.nomeLivro = (String)dr["nomeLivro"];
            livro.imagemLivro = (String)dr["imagemLivro"];
        }
        return livro;
    }
}

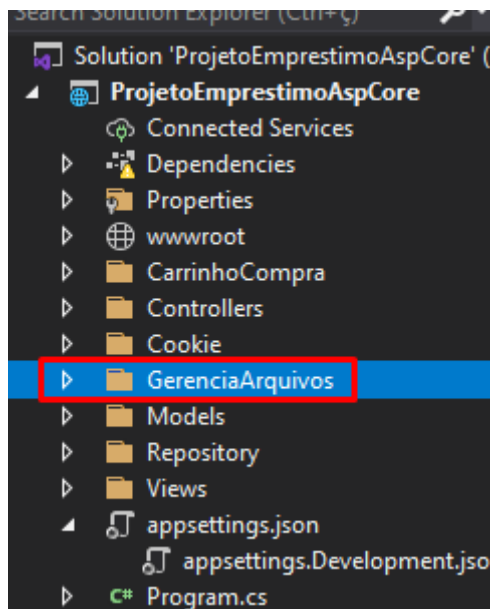
```



Vamos criar um método para obter todos os livros na classe **LivroRepository**

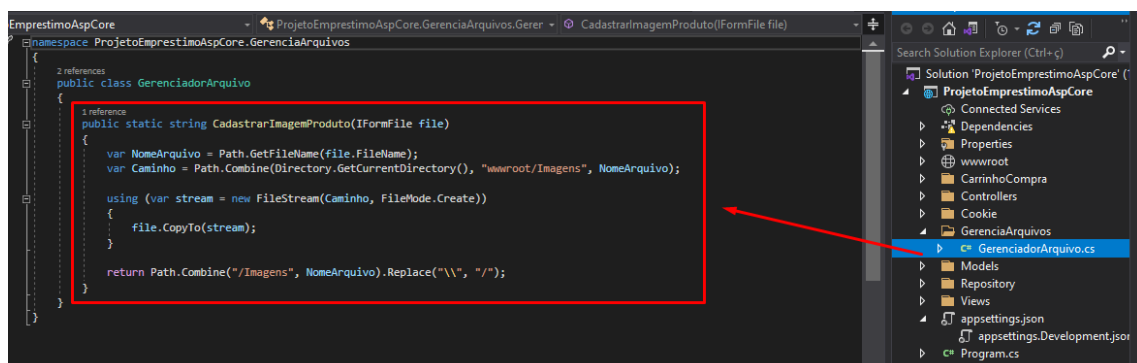


Vamos criar uma pasta na raiz do projeto com o nome **GerenciaArquivos**



Dentro vamos colocar um classe **GerenciadorArquivo**

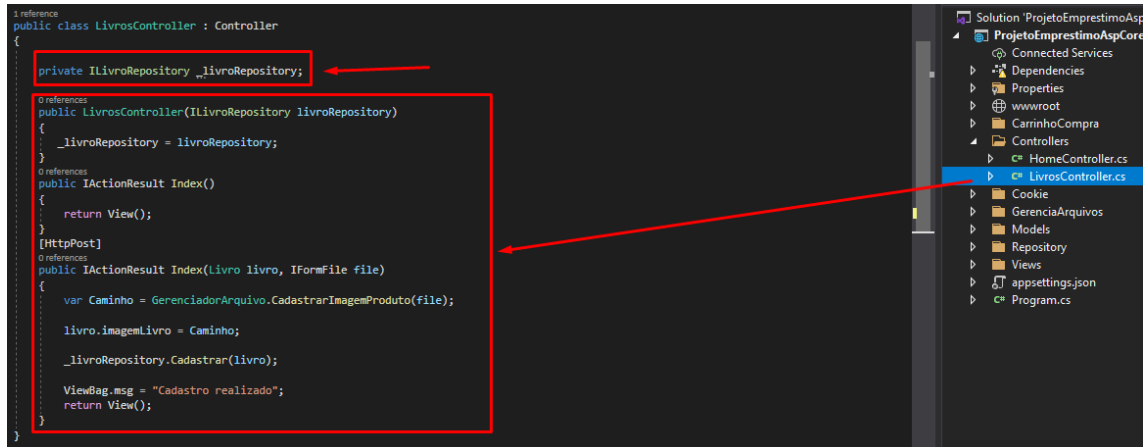
Dentro desta classe vamos criar um método para criar o caminho da nossa pasta para salvar as imagens



Vamos adicionar um LivroController

No Controller vamos primeiramente injetar o livro **ILivroRepository** e usar no construtor

Depois vamos adicionar um método que cadastra o livro



```
using Microsoft.AspNetCore.Mvc;

public class LivrosController : Controller
{
    private ILivroRepository _livroRepository;

    public LivrosController(ILivroRepository livroRepository)
    {
        _livroRepository = livroRepository;
    }

    public IActionResult Index()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Index(Livro livro, IFormFile file)
    {
        var Caminho = GerenciadorArquivo.CadastrarImagemProduto(file);
        livro.imagemLivro = Caminho;
        _livroRepository.Cadastrar(livro);

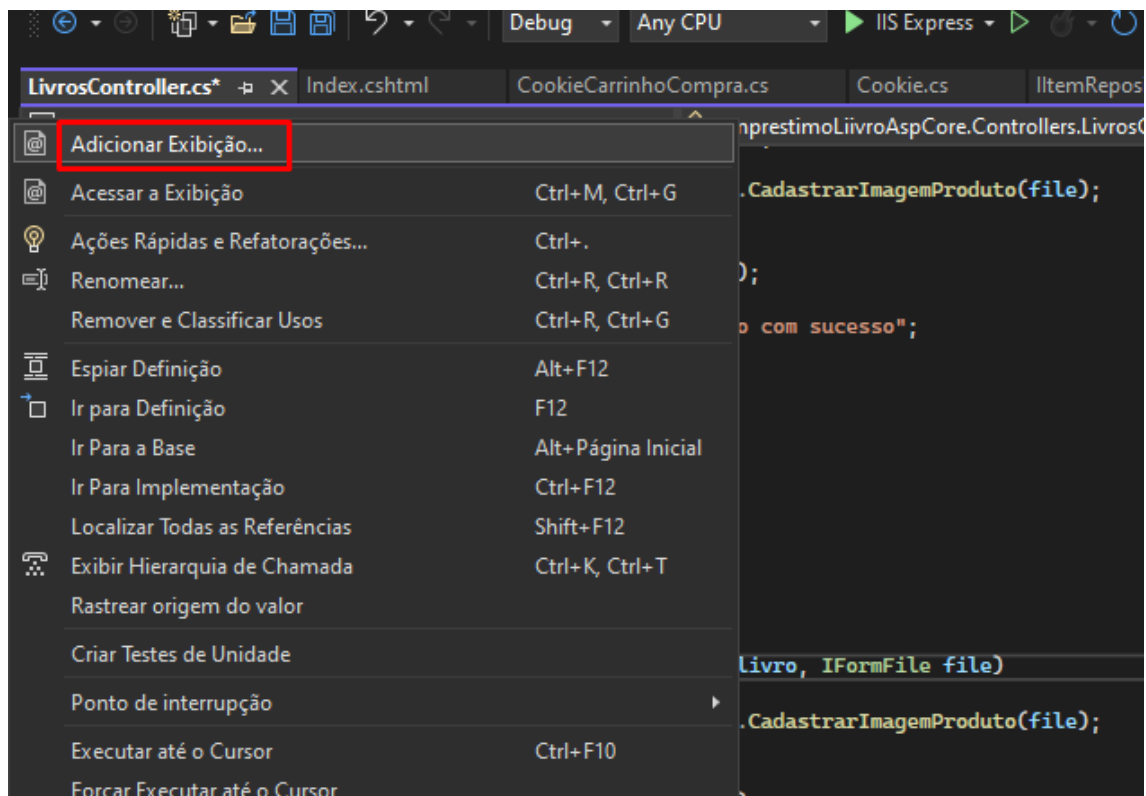
        ViewBag.msg = "Cadastro realizado";
        return View();
    }
}
```

Vamos criar a view que cadastra o livro usando a Index

```
1 @model ProjetoEmprestimoAspCore.Models.Livro
2 @if
3     ViewBag.Title = "Cadastrar Novo Livro";
4 }
5 <h2 style="text-align:center;">Novo Livro</h2>
6
7 <div class="center">
8     <div class="form-horizontal">
9         <div class="form-group">
10             <div class="form-group space-around-flex">
11                 <div class="">
12                     @Html.LabelFor(model => model.nomeLivro, htmlAttributes: new { @class = "control-label col-md-2" })
13                 </div>
14                 <div class="col-md-10">
15                     @Html.EditorFor(model => model.nomeLivro, new { htmlAttributes = new { @class = "form-control" } })
16                     @Html.ValidationMessageFor(model => model.nomeLivro, "", new { @class = "text-danger" })
17                 </div>
18             </div>
19         </div>
20         <div class="form-group center">
21             <div>
22                 <label class="control-label col-md-2" for="file">Capa Livro</label>
23             </div>
24             <div class="col-md-10">
25                 <input type="file" name="file" id="file" /> <br />
26             </div>
27         </div>
28     </div>
29
30     <div class="form-group center">
31         <div class="col-md-10">
32             @ViewBag.msg <br />
33         </div>
34     </div>
35
36     <div class="form-group">
37         <div class="col-md-offset-2 col-md-10">
38             <input type="submit" value="Salvar Produto" class="btn btn-success" />
39         </div>
40     </div>
41 </div>
```

Vamos adicionar a interface como serviço na classe Program.cs

```
Program.cs*  HomeController.cs  LivrosController.cs  LivroRepository.cs
ProjetoEmprestimoAspCore
4 using ProjetoEmprestimoAspCore.Repository.Contrato;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 // Add services to the container.
9 builder.Services.AddControllersWithViews();
10
11 builder.Services.AddHttpContextAccessor();
12
13 // Adicionar um serviço para Injetar os metodos no Controller
14 builder.Services.AddScoped<ILivroRepository, LivroRepository>();
15
```



## Adicionar Exibição do Razor

Nome da exibição:

Modelo:

Classe do modelo:

Opções

- ☐ Criar como um modo de exibição parcial
- ☒ Bibliotecas de scripts de referência
- ☒ Usar uma página de layout

...

(Deixe em branco se ele estiver definido em um arquivo Razor \_viewstart)

```
ml CadLivre.cshtml LivrosController.cs Index.cshtml CookieCarrinhoCompra.cs Cookie.cs ItemRepository
timoliivoAspCore
@model EmprestimoLivroAspCore.Models.Livro

@{
    ViewData["Title"] = "CadLivre";
}

<h1>CadLivre</h1>

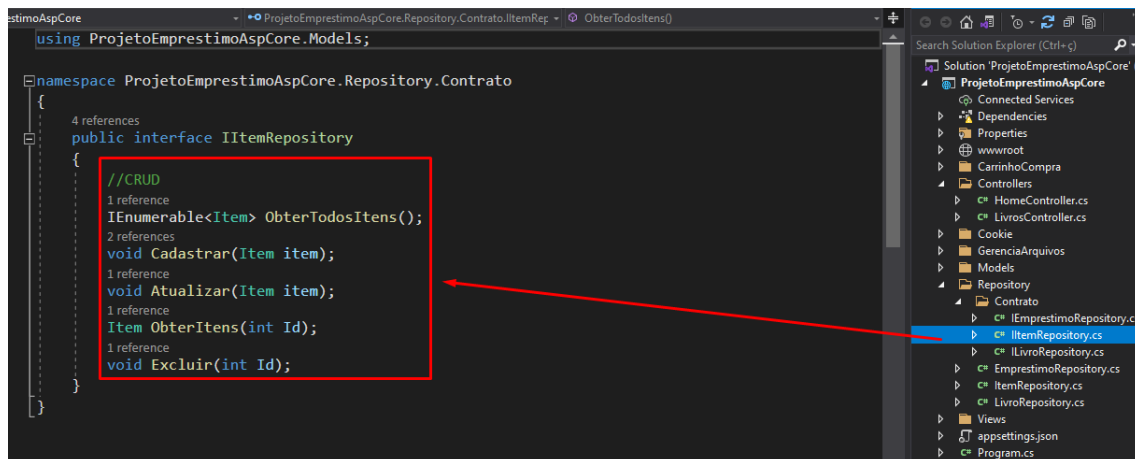
<h4>Livre</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="CadLivre" asp-controller="Livros" method="post" enctype="multipart/form-data">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <input asp-for="codLivre" type="hidden" class="form-control" />
                <span asp-validation-for="codLivre" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="nomeLivre" class="control-label"></label>
                <input asp-for="nomeLivre" class="form-control" />
                <span asp-validation-for="nomeLivre" class="text-danger"></span>
            </div>
        </form>
    </div>
    <div class="col-md-4">
        <div class="form-group">
            <input type="file" name="file" id="file" /> <br />
            <span asp-validation-for="imagemLivre" class="text-danger"></span>
        </div>
        @*VAMOS ADD ESTA MENSAGEM*@
        @ViewBag.msg
        <br />
        <div class="form-group">
            <input type="submit" value="Cadastrar" class="btn btn-success" />
        </div>
    </div>
</div>

</div>
<div>
    <a asp-action="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

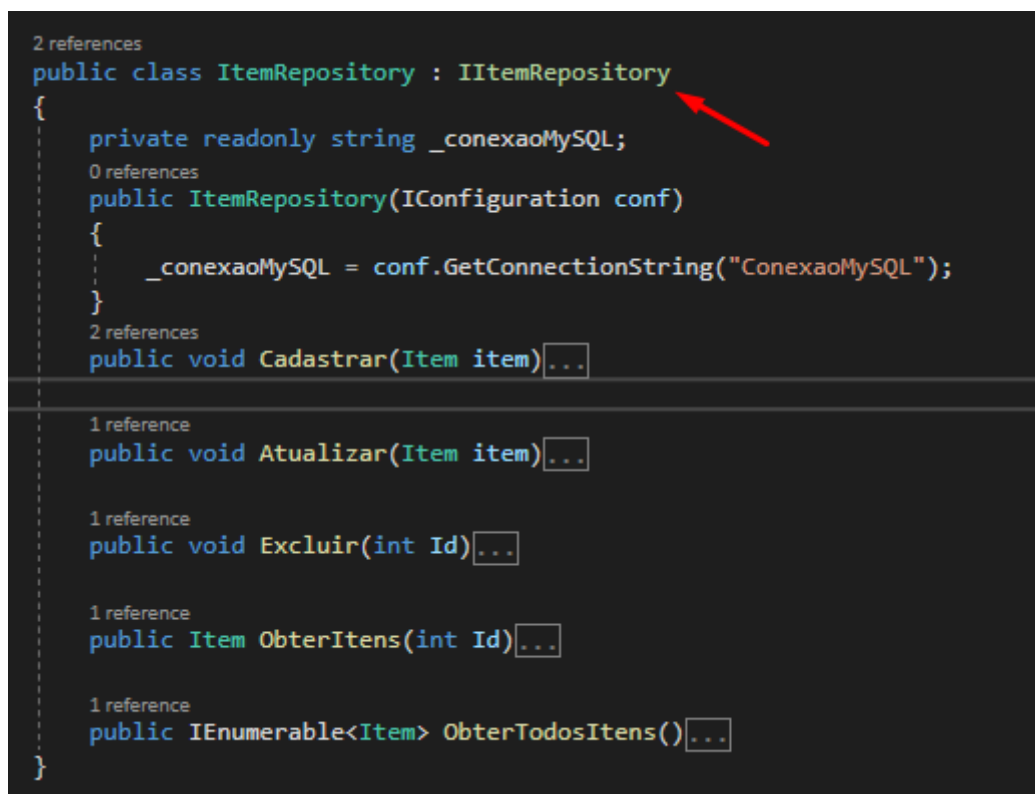
Vamos criar a interface de ItemRepository

Clicando com o botão direito sobre a pasta Contract, depois em adicionar –  
Classe



Vamos criar uma classe **ItemRepository**

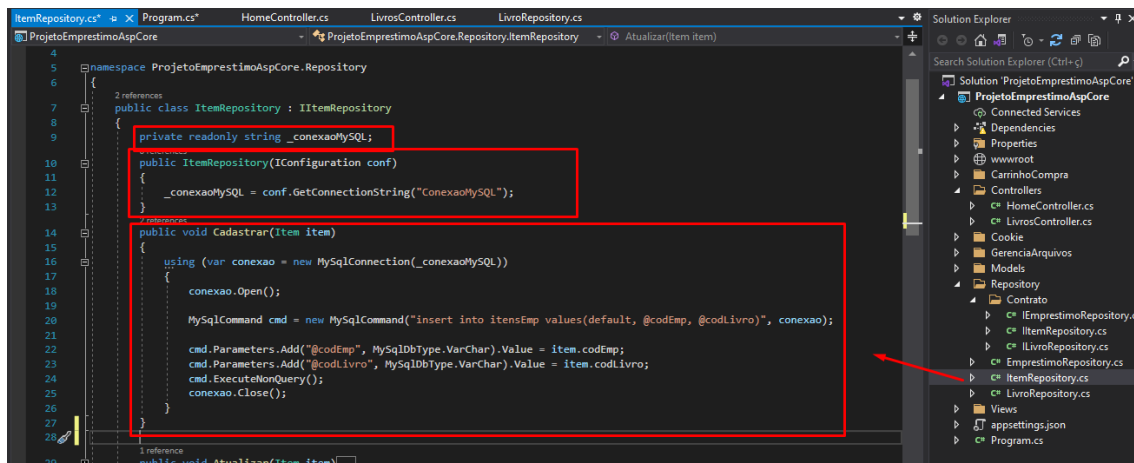
Vamos colocar um Extends para a Interface e implementar os métodos



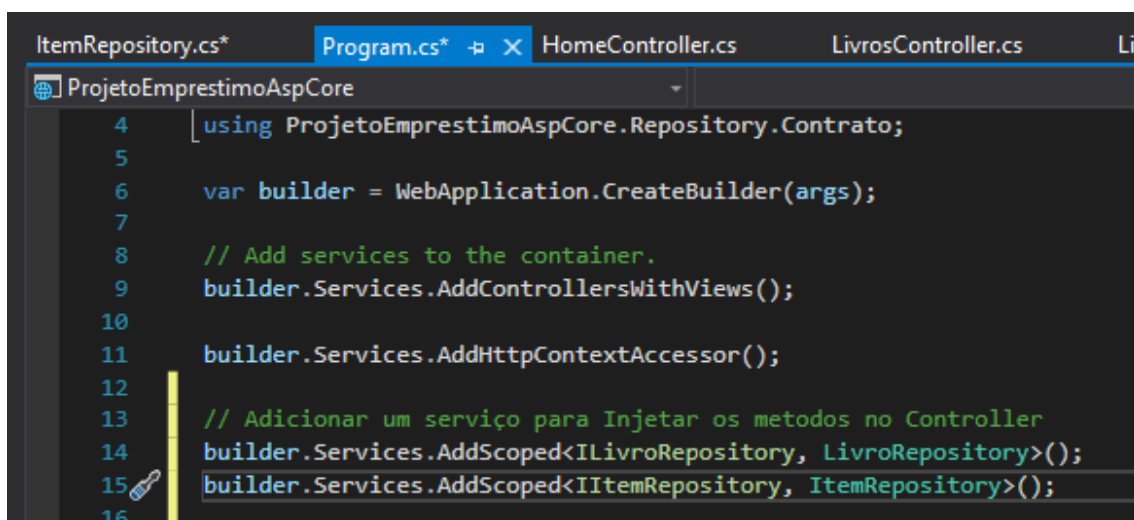
Vamos primeiramente criar a conexão colocando a propriedade privada,

Depois injetamos no construtor da classe

Depois vamos reescrever o método cadastrar

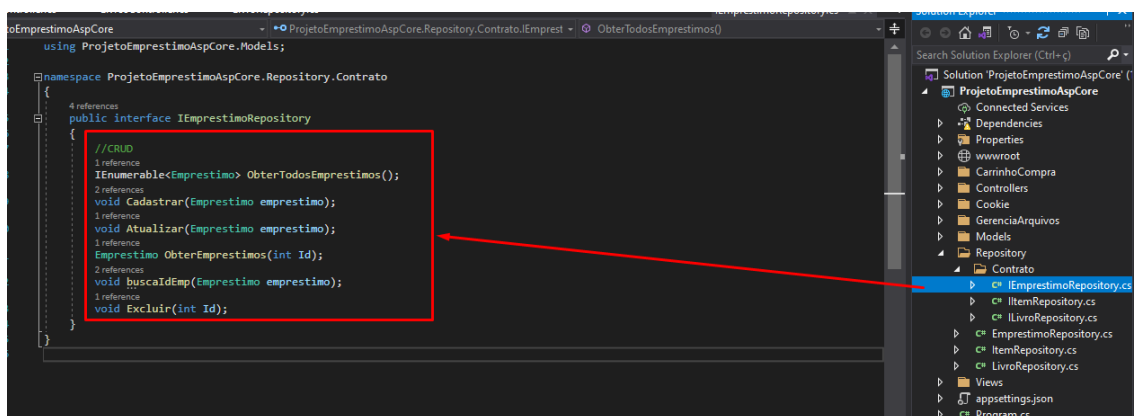


Vamos adicionar a interface como serviço na classe Program.cs



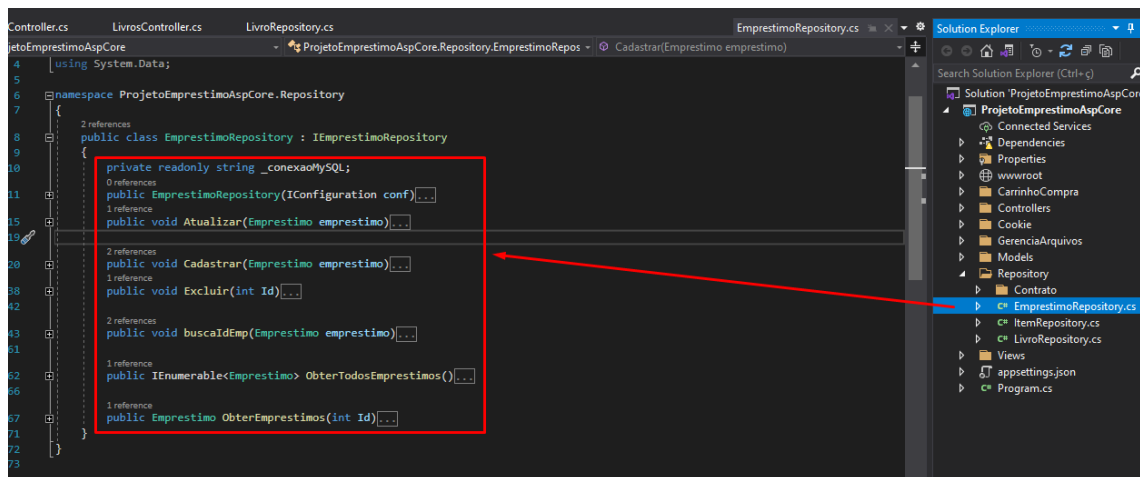
Vamos criar a interface de IEmprestimoRepository

Clicando com o botão direito sobre a pasta Contract, depois em adicionar – Classe



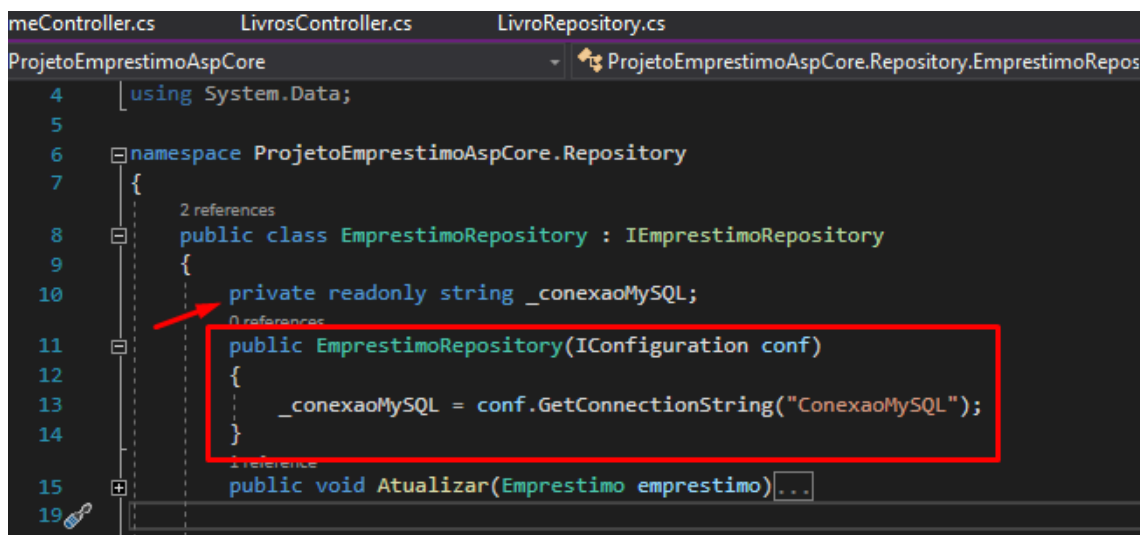


Vamos criar a classe concreta e implementar os métodos da interface



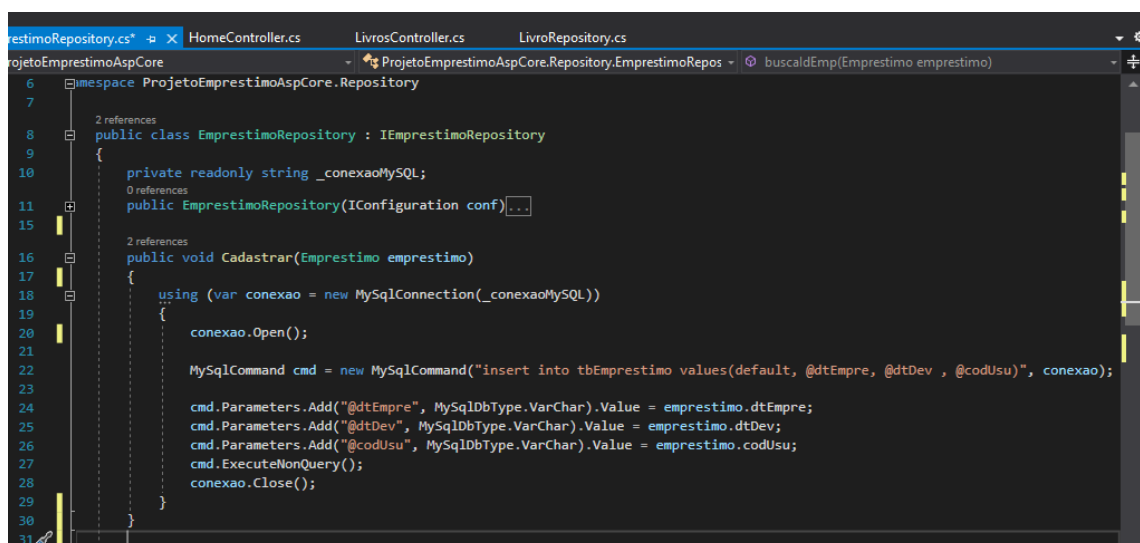
```
1 using System.Data;
2
3 namespace ProjetoEmprestimoAspCore.Repository
4 {
5     2 references
6     public class EmprestimoRepository : IEmprestimoRepository
7     {
8         private readonly string _conexaoMySQL;
9         0 references
10        public EmprestimoRepository(IConfiguration conf) {...}
11        1 reference
12        public void Atualizar(Emprestimo emprestimo) {...}
13
14        2 references
15        public void Cadastrar(Emprestimo emprestimo) {...}
16        1 reference
17        public void Excluir(int Id) {...}
18
19        2 references
20        public void buscaIdEmp(Emprestimo emprestimo) {...}
21
22        1 reference
23        public IEnumerable<Emprestimo> ObterTodosEmprestimos() {...}
24
25        1 reference
26        public Emprestimo ObterEmprestimos(int Id) {...}
27    }
28 }
```

Vamos colocar a propriedade privada da conexão e depois injetar no construtor



```
1 using System.Data;
2
3 namespace ProjetoEmprestimoAspCore.Repository
4 {
5     2 references
6     public class EmprestimoRepository : IEmprestimoRepository
7     {
8         private readonly string _conexaoMySQL;
9         0 references
10        public EmprestimoRepository(IConfiguration conf)
11        {
12            _conexaoMySQL = conf.GetConnectionString("ConexaoMySQL");
13        }
14        1 reference
15        public void Atualizar(Emprestimo emprestimo) {...}
16
17        2 references
18        public void Cadastrar(Emprestimo emprestimo) {...}
19        1 reference
20        public void Excluir(int Id) {...}
21
22        2 references
23        public void buscaIdEmp(Emprestimo emprestimo) {...}
24
25        1 reference
26        public IEnumerable<Emprestimo> ObterTodosEmprestimos() {...}
27
28        1 reference
29        public Emprestimo ObterEmprestimos(int Id) {...}
30    }
31 }
```

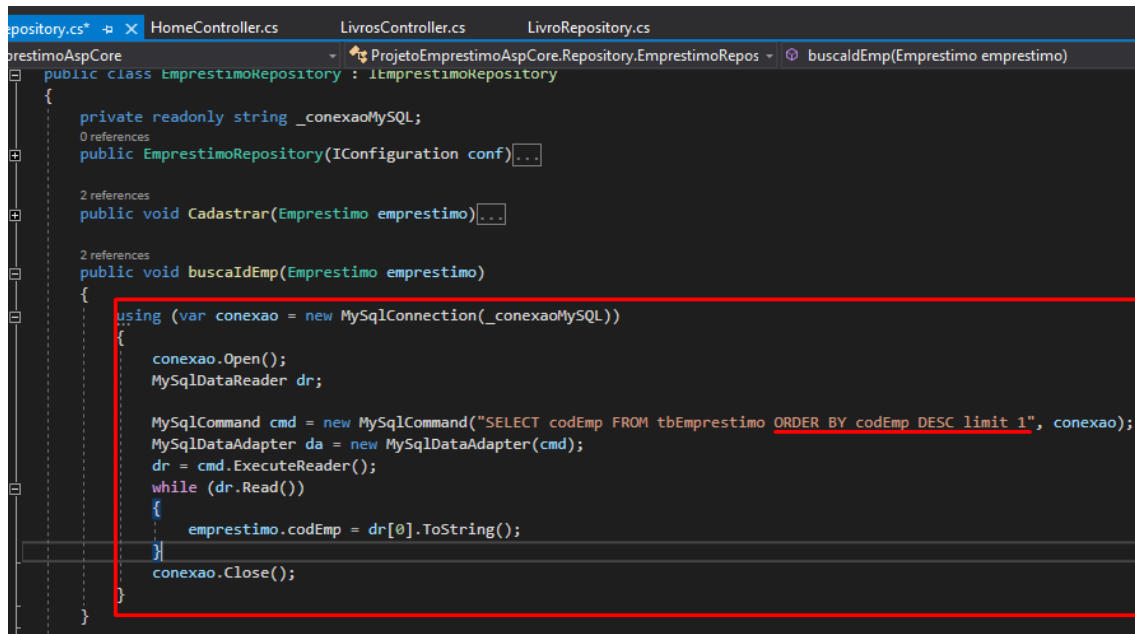
Vamos reescrever o método para cadastrar o empréstimo.



```
1 using System.Data;
2
3 namespace ProjetoEmprestimoAspCore.Repository
4 {
5     2 references
6     public class EmprestimoRepository : IEmprestimoRepository
7     {
8         private readonly string _conexaoMySQL;
9         0 references
10        public EmprestimoRepository(IConfiguration conf) {...}
11
12        2 references
13        public void Cadastrar(Emprestimo emprestimo)
14        {
15            using (var conexao = new MySqlConnection(_conexaoMySQL))
16            {
17                conexao.Open();
18
19                MySqlCommand cmd = new MySqlCommand("insert into tbEmprestimo values(default, @dtEmpre, @dtDev , @codUsu)", conexao);
20
21                cmd.Parameters.Add("@dtEmpre", MySqlDbType.VarChar).Value = emprestimo.dtEmpre;
22                cmd.Parameters.Add("@dtDev", MySqlDbType.VarChar).Value = emprestimo.dtDev;
23                cmd.Parameters.Add("@codUsu", MySqlDbType.VarChar).Value = emprestimo.codUsu;
24                cmd.ExecuteNonQuery();
25                conexao.Close();
26            }
27        }
28
29        2 references
30        public void buscaIdEmp(Emprestimo emprestimo) {...}
31
32        1 reference
33        public IEnumerable<Emprestimo> ObterTodosEmprestimos() {...}
34
35        1 reference
36        public Emprestimo ObterEmprestimos(int Id) {...}
37    }
38 }
```

Vamos reescrever o método para buscar o empréstimo por ID.

Esse método pega o ultimo ID que foi inserido no banco



```
public class EmprestimoRepository : IEmprestimoRepository
{
    private readonly string _conexaoMySQL;

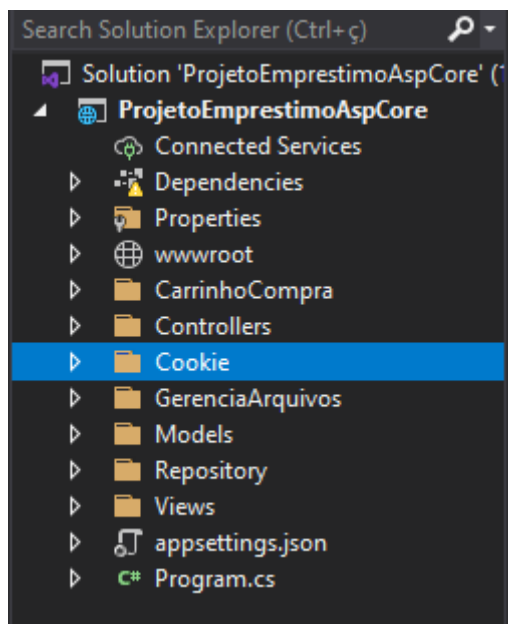
    public EmprestimoRepository(IConfiguration conf)
    {
        _conexaoMySQL = conf.GetConnectionString("MySQL");
    }

    public void Cadastrar(Emprestimo emprestimo)
    {
        // ...
    }

    public void buscaIdEmp(Emprestimo emprestimo)
    {
        using (var conexao = new MySqlConnection(_conexaoMySQL))
        {
            conexao.Open();
            MySqlDataReader dr;

            MySqlCommand cmd = new MySqlCommand("SELECT codEmp FROM tbEmprestimo ORDER BY codEmp DESC limit 1", conexao);
            MySqlDataAdapter da = new MySqlDataAdapter(cmd);
            dr = cmd.ExecuteReader();
            while (dr.Read())
            {
                emprestimo.codEmp = dr[0].ToString();
            }
            conexao.Close();
        }
    }
}
```

Vamos criar uma pasta na raiz do projeto com nome Cookie



Vamos criar uma classe para manipular os cookie

Primeiramente vamos adicionar

```
private IHttpContextAccessor _context;
private IConfiguration _configuration;
```

Depois injetar no método construtor

```
0 references
public Cookie(IHttpContextAccessor context, IConfiguration configuration)
{
    _context = context;
    _configuration = configuration;
}
```

```
ProjetoEmprestimoAspCore - ProjetoEmprestimoAspCore.Cookie.Cookie - _configuration
1 using Fersoft.NETCore.Cipher;
2
3 namespace ProjetoEmprestimoAspCore.Cookie
4 {
5     4 references
6     public class Cookie
7     {
8         private IHttpContextAccessor _context;
9         private IConfiguration _configuration;
10
11     0 references
12     public Cookie(IHttpContextAccessor context, IConfiguration configuration)
13     {
14         _context = context;
15         _configuration = configuration;
16     }
17
18     /*
19     * CRUD - Cadastrar/Atualizar/Consultar/Remover - RemoverTodos/Exist
20     */
21     2 references
22     public void Cadastrar(string Key, string Valor)...
23     0 references
24     public void Atualizar(string Key, string Valor)...
25     3 references
26     public void Remover(string Key)...
27     1 reference
28     public string Consultar(string Key, bool Cript = true)...
29     4 references
30     public bool Existe(string Key)...
31     0 references
}
```

```
// Cadastrar Cookie
2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Cadastrar(string Key, string Valor)
{
    CookieOptions Options = new CookieOptions();
    Options.Expires = DateTime.Now.AddDays(7);
    Options.IsEssential = true;

    _context.HttpContext.Response.Cookies.Append(Key, Valor, Options);
}

// Deleta Cookie
2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Remover(string Key)
{
    _context.HttpContext.Response.Cookies.Delete(Key);
}

// Consulta Cookie
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public string Consultar(string Key, bool Cript = true)
{
    var valor = _context.HttpContext.Request.Cookies[Key];
    return valor;
}
```

```

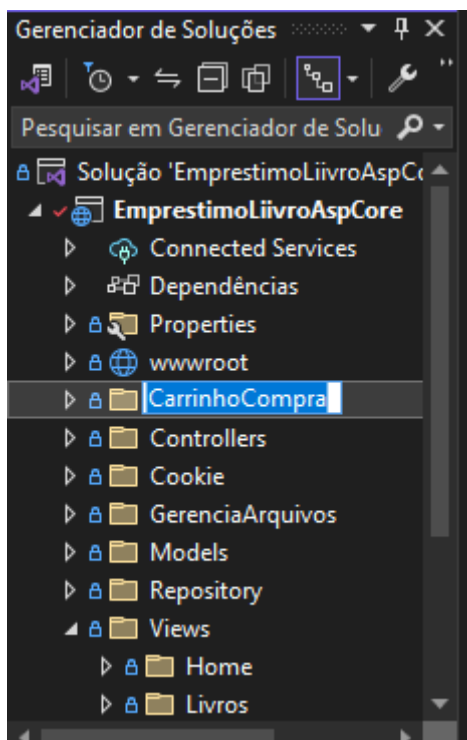
4 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public bool Existe(string Key)
{
    if (_context.HttpContext.Request.Cookies[Key] == null)
    {
        return false;
    }

    return true; ;
}

0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Atualizar(string Key, string Valor)
{
    if (Existe(Key))
    {
        Remover(Key);
    }
    Cadastrar(Key, Valor);
}

```

Vamos criar uma pasta **CarrinhoCompra**



Dentro desta pasta vamos criar uma classe **CookieCarrinhoCompra**

Depois vamos colocar duas propriedades privada

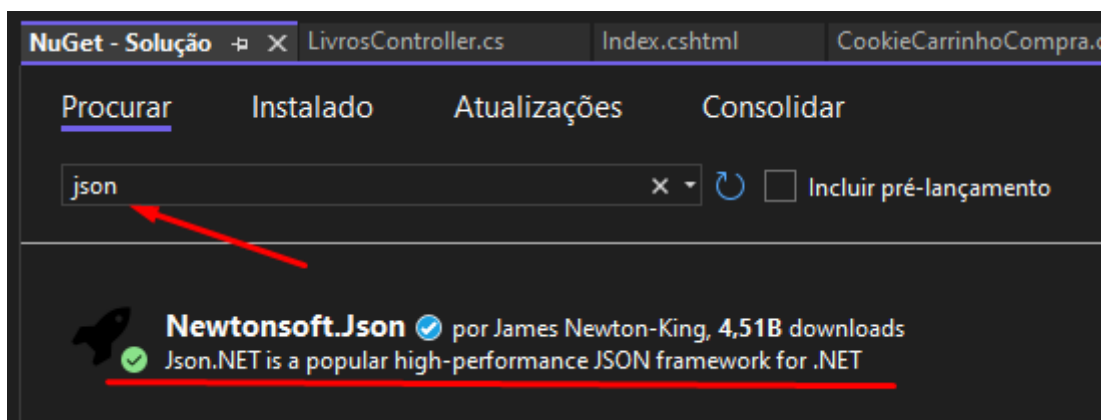
A primeira é a chave e depois injetar a classe cookie

```
//criar uma chave
private string Key = "Carrinho.Compras";
private Cookie.Cookie _cookie;

0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public CookieCarrinhoCompra(Cookie.Cookie cookie)
{
    _cookie = cookie;
}
```

Vamos criar os métodos para adicionar os itens no carrinho de compras

Antes vamos usar uma biblioteca Newtonsoft.Json que vai permitir Serializar Deserializar Objeto.



Método para salvar e consultar

```
//Salvar
3 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Salvar(List<Livro> Lista)
{
    string Valor = JsonConvert.SerializeObject(Lista);
    _cookie.Cadastrar(Key, Valor);
}

//Consulta
5 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public List<Livro> Consultar()
{
    if (_cookie.Existe(Key))
    {
        string valor = _cookie.Consultar(Key);
        return JsonConvert.DeserializeObject<List<Livro>>(valor);
    }
    else
    {
        return new List<Livro>();
    }
}
```

## Método para cadastrar o item

```
//Cadastrar
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Cadastrar(Livro item)
{
    List<Livro> Lista;
    if (_cookie.Existe(Key))
    {
        Lista = Consultar();
        var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

        if (ItemLocalizado == null)
        {
            Lista.Add(item);
        }
        else
        {
            ItemLocalizado.quantidade = ItemLocalizado.quantidade + 1;
        }
    }
    else
    {
        Lista = new List<Livro>();
        Lista.Add(item);
    }
    // Criar o método salvar
    Salvar(Lista);
}
```

## Método para atualizar os itens e remover

```
//Atualiza
0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public void Atualizar(Livro item)
{
    var Lista = Consultar();
    var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

    if (ItemLocalizado != null)
    {
        ItemLocalizado.quantidade = item.quantidade + 1;
        Salvar(Lista);
    }
}

// remove item
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Remover(Livro item)
{
    var Lista = Consultar();
    var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

    if (ItemLocalizado != null)
    {
        Lista.Remove(ItemLocalizado);
        Salvar(Lista);
    }
}
```

Método para verificar os itens e remover todos

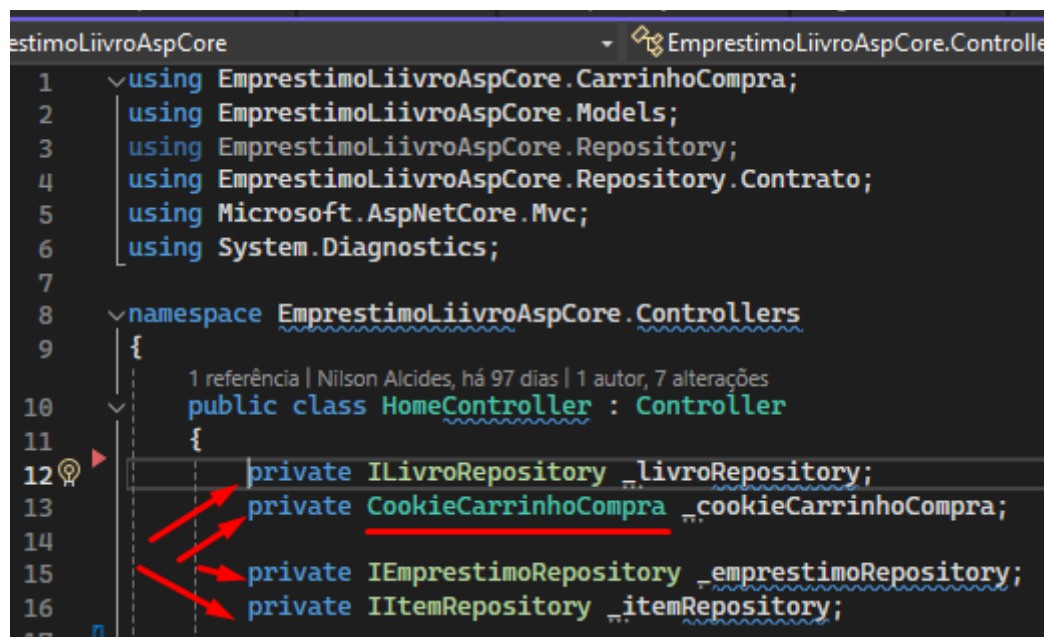
```
// Verifica se existe
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public bool Existe(string Key)
{
    if (_cookie.Existe(Key))
    {
        return false;
    }

    return true;
}

// Remove todos itens do carrinho
1 referência | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public void RemoverTodos()
{
    _cookie.Remover(Key);
}
```

Agora vamos exibir os itens na Home Index

Acessando o HomeController vamos criar as propriedades privadas para injetar no método construtor da classe



```
EmprestimoLivroAspCore
EmprestimoLivroAspCore.Controllers

1  using EmprestimoLivroAspCore.CarrinhoCompra;
2  using EmprestimoLivroAspCore.Models;
3  using EmprestimoLivroAspCore.Repository;
4  using EmprestimoLivroAspCore.Repository.Contrato;
5  using Microsoft.AspNetCore.Mvc;
6  using System.Diagnostics;
7
8  namespace EmprestimoLivroAspCore.Controllers
9  {
10     1 referência | Nilson Alcides, há 97 dias | 1 autor, 7 alterações
11     public class HomeController : Controller
12     {
13         private ILivroRepository _livroRepository;
14         private CookieCarrinhoCompra _cookieCarrinhoCompra;
15         private IEmprestimoRepository _emprestimoRepository;
16         private IItemRepository _itemRepository;
17     }
```

Vamos criar o construtor do HomeController

```
namespace EmprestimoLivroAspNetCore.Controllers
{
    1 referência | Nilson Alcides, há 97 dias | 1 autor, 7 alterações
    public class HomeController : Controller
    {
        private IItemRepository _itemRepository;
        private IEmprestimoRepository _emprestimoRepository;
        private CookieCarrinhoCompra _cookieCarrinhoCompra;
        private ILivroRepository _livroRepository;

        0 referências | 0 alterações | 0 autores, 0 alterações
        public HomeController(ILivroRepository livroRepository, CookieCarrinhoCompra cookieCarrinhoCompra,
            IEmprestimoRepository emprestimoRepository, IItemRepository itemRepository)
        {
            _livroRepository = livroRepository;
            cookieCarrinhoCompra = cookieCarrinhoCompra;
            emprestimoRepository = emprestimoRepository;
            itemRepository = itemRepository;
        }
    }
}
```

Na action index vamos colocar o método que retorna os livros.

```
//Exibe os livros na Index
0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public IActionResult Index()
{
    return View(_livroRepository.ObterTodosLivros());
}
```

Vamos criar a Index que exibe os livros

```
1 @model IEnumerable<EmprestimoLivroAspNetCore.Models.Livro>
2
3 @{
4     ViewBag.Title = "Biblioteca";
5 }
6 <div class="row">
7     <div class="row center">
8         <div class="col-lg-9">
9             <div class="row">
10                 @foreach (var item in Model)
11                 {
12                     <div class="col-lg-3 col-md-6 mb-4" style="margin:2% 4%">
13                         <div class="card h-100">
14                             @if (@item.imagemLivro != null)
15                             {
16                                 
18                             }
19                             else
20                             {
21                                 
23                             }
24                         </div>
25                         <div class="card-body">
26                             <h4 class="card-title">
27                                 <a href="@item.imagemLivro">@item.nomeLivro</a>
28                             </h4>
29                             <span>Código: @item.codLivro</span>
30                         </div>
31                         <div class="card-footer space-between flex" style="width:90%">
32                             <div>
33                                 @using (Html.BeginForm("AdicionarItem", "Home", FormMethod.Post))
34                                 {
35                                     <input type="hidden" name="id" value=@item.codLivro>
36                                     <input type="hidden" name="quantidade" value="1">
37                                     <button type="submit" class="btn btn-default btn-acao-primario">Adicionar</button>
38                                 }
39                             </div>
40                         </div>
41                     </div>
42                 }
43             </div>
44         </div>
45     </div>
46 </div>
```

Vamos criar uma Action AdicionarItem



```

32 //Item ID = ID Produto
33 0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
34 public IActionResult AdicionarItem(int id)
35 {
36     Livro produto = _livroRepository.ObterLivros(id);
37
38     if (produto == null)
39     {
40         return View("NaoExisteItem");
41     }
42     else
43     {
44         var item = new Livro()
45         {
46             codLivro = id,
47             quantidade = 1,
48             imagemLivro = produto.imagemLivro,
49             nomeLivro = produto.nomeLivro
50         };
51         _cookieCarrinhoCompra.Cadastrar(item);
52
53         return RedirectToAction(nameof(Carrinho));
54     }
55

```

//Carrinho de compra

2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração

public IActionResult Carrinho()

{

return View(\_cookieCarrinhoCompra.Consultar());

}

//Remover itens do carrinho

0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração

public IActionResult RemoverItem(int id)

{

\_cookieCarrinhoCompra.Remover(new Livro() { codLivro = id });

return RedirectToAction(nameof(Carrinho));

}

```

DateTime data;
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public IActionResult SalvarCarrinho(Emprestimo emprestimo)
{
    List<Livro> carrinho = _cookieCarrinhoCompra.Consultar();

    Emprestimo mdE = new Emprestimo();
    Item mdI = new Item();

    data = DateTime.Now.ToLocalTime();

    mdE.dtEmpre = data.ToString("dd/MM/yyyy");
    mdE.dtDev = data.AddDays(7).ToString();
    mdE.codUsu = "1";
    _emprestimoRepository.Cadastrar(mdE);

    _emprestimoRepository.buscaIdEmp(emprestimo);

    for (int i = 0; i < carrinho.Count; i++)
    {
        mdI.codEmp = Convert.ToInt32(emprestimo.codEmp);
        mdI.codLivro = Convert.ToString(carrinho[i].codLivro);

        _itemRepository.Cadastrar(mdI);
    }

    _cookieCarrinhoCompra.RemoverTodos();
    return RedirectToAction("confEmp");
}

```

Confirmar empréstimo

```

0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public IActionResult confEmp()
{
    return View();
}

```

Vamos criar as Views

Primeiro o carrinho

```

1  ✓ @model IEnumerable<EmprestimoLivroAspCore.Models.Livro>
2
3  @{
4      ViewData["Title"] = "Index";
5      decimal Subtotal = 0;
6  }
7
8  <div class="container">
9      <section id="order">
10         <br />
11         <br />
12         <h2 class="title-doc">Carrinho de compras</h2>
13         <div class="alert alert-danger" style="display: none;" role="alert">
14         </div>
15
16         <div id="code_cart">
17             <div class="card">
18                 <table class="table table-hover shopping-cart-wrap">
19                     <thead class="text-muted">
20                         <tr>
21                             <th scope="col">Produto</th>
22                             <th scope="col" width="120">Quantidade</th>
23                             <th scope="col" width="120"></th>
24                             <th scope="col" width="200" class="text-right">Ação</th>
25                         </tr>
26                     </thead>

```

```

</thead>
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>
                <figure class="media">
                    <div class="img-wrap">
                        
                    </div>
                    <figcaption class="media-body">
                        <h6 class="title text-truncate">@item.nomeLivro</h6>
                        <dl class="dlist-inline small">
                            <dt>Codigo Livro: @item.codLivro </dt>
                            <dd>@* @item.codLivro *@</dd>
                        </dl>
                    </figcaption>
                </figure>
            </td>

```

```

            <td>
                <div class="col-auto">
                    <div class="input-group mb-2 control-inline">
                        <input type="hidden" class="inputProdutoId" value="@item.codLivro" />
                        <input type="hidden" class="inputQuantidadeEstoque" value="@item.quantidade" />
                        <input type="hidden" class="inputValorUnitario" value="R$ 00,00" />
                        <input type="text" style="width: 60px; text-align: center; readonly="readonly"
                            class="form-control inputQuantidadeProdutoCarrinho" value="@item.quantidade">
                    </div>
                </td>
            <td>
                </td>
            <td class="text-right">
                <a asp-controller="Home" asp-action="RemoverItem" asp-route-id="@item.codLivro" class="btn btn-outline-danger"> x Remove</a>
            </td>
        </tr>
    }
</tbody>
</table>
</div>
</section>
<br />
</div>

```

```
72 |         <br />
73 |         <br />
74 |     </div>
75 |
76 | <div class="container">
77 |     <br />
78 |     <br />
79 |     <div class="row">
80 |         <div class="col-md-12">
81 |             Deseja adicionar outro? <a asp-controller="Home" asp-action="Index">Clique aqui</a> para continuar ...
82 |         </div>
83 |         <br />
84 |         <br />
85 |         <div class="offset-md-4 col-md-4">
86 |             <a class="btn btn-outline-primary btn-continuar-comprando" asp-action="SalvarCarrinho" asp-controller="Home">
87 |                 Finalizar o Empréstimo
88 |             </a>
89 |         </div>
90 |     </div>
91 | </div>
92 |
```

```

@model IEnumerable<EmprestimoLivro.AspNetCore.Models.Livro>
@{ ViewData["Title"] = "Index"; decimal Subtotal = 0; }

<div class="container">
    <section id="order">
        <br />
        <br />
        <h2 class="title-doc">Carrinho de compras</h2>
        <div class="alert alert-danger" style="display: none;" role="alert"></div>
        <div id="code_cart">
            <div class="card">
                <table class="table table-hover shopping-cart-wrap">
                    <thead class="text-muted">
                        <tr>
                            <th scope="col">Produto</th>
                            <th scope="col" width="120">Quantidade</th>
                            <th scope="col" width="120"></th>
                            <th scope="col" width="200" class="text-right">Ação</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var item in Model)
                        {
                            <tr>
                                <td>
                                    <figure class="media">
                                        <div class="img-wrap">
                                            
                                        </div>
                                        <figcaption class="media-body">
                                            <h6 class="title text-truncate">@item.nomeLivro</h6>
                                            <dl class="dlist-inline small">
                                                <dt>Codigo Livro: @item.codLivro </dt>
                                                <dd>@* @item.codLivro *@</dd>
                                            </dl>
                                        </figcaption>
                                    </figure>
                                </td>
                                <td>
                                    <div class="col-auto">
                                        <div class="input-group mb-2 control-inline">
                                            <input type="hidden" class="inputProdutoId" value="@item.codLivro" />
                                            <input type="hidden" class="inputQuantidadeEstoque" value="@item.quantidade" />
                                            <input type="hidden" class="inputValorUnitario" value="R$ 00,00" />
                                            <input type="text" style="width: 60px; text-align: center;" readonly="readonly" class="form-control" value="@item.quantidade" />
                                        </div>
                                    </div>
                                </td>
                                <td class="text-right">
                                    <a asp-controller="Home" asp-action="RemoverItem" asp-route-id="@item.codLivro" class="btn btn-outline-danger"> × Remove</a>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>
        </section>
        <br />
        <br />
    </div>
    <div class="container">
        <br />
        <br />
        <div class="row">
            <div class="col-md-12">
                Deseja adicionar outro? <a asp-controller="Home" asp-action="Index">Clique aqui</a> para continuar ...
            </div>
            <div class="offset-md-4 col-md-4">
                <a class="btn btn-outline-primary btn-continuar-comprando" asp-action="SalvarCarrinho" asp-controller="Home">Finalizar o Empréstimo</a>
            </div>
        </div>
    </div>
</div>

```

View para confirmar o empréstimo

```

1  @{
2      ViewBag.Title = "confEmp";
3  }
4
5  <h2>Empréstimo Finalizado</h2>
6

```

Vamos adicionar os serviços na classe Program.cs

```
1 using EmprestimoLivroAspCore.Repository.Contrato;
2 using EmprestimoLivroAspCore.Repository;
3 using EmprestimoLivroAspCore.GerenciaArquivos;
4 using System.Net;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 // Add services to the container.
9 builder.Services.AddControllersWithViews();
10
11 builder.Services.AddHttpContextAccessor();
12
13 //Adicionar a interface como serviço
14 builder.Services.AddScoped<ILivroRepository, LivroRepository>();
15 builder.Services.AddScoped<IEmprestimoRepository, EmprestimoRepository>();
16 builder.Services.AddScoped<IItemRepository, ItemRepository>();
17
18 builder.Services.Configure<CookiePolicyOptions>(options =>
19 {
20     options.CheckConsentNeeded = context => true;
21     options.MinimumSameSitePolicy = SameSiteMode.None;
22 });
23
```

HttpContextAccessor. Você pode usá-lo para obter detalhes sobre solicitações recebidas, incluindo cabeçalhos, cookies, parâmetros de consulta e declarações de usuários. Ao obter acesso à propriedade "Resposta", você também pode alterar a resposta HTTP

Adicionando as classes como

```
1 // Corrigir problema com TEMPDATA para aumentar o tempo de duração
2 builder.Services.AddDistributedMemoryCache();
3 builder.Services.AddSession(options =>
4 {
5     // Set a short timeout for easy testing.
6     options.IdleTimeout = TimeSpan.FromSeconds(900);
7     options.Cookie.HttpOnly = true;
8     // Deixar informado para o navegador que a sessão é essencial
9     options.Cookie.IsEssential = true;
10 });
11 builder.Services.AddMvc().AddSessionStateTempDataProvider();
12
13 builder.Services.AddMemoryCache(); // Guardar os dados na memoria
14
15 //Add Gerenciador Arquivo como serviços
16 builder.Services.AddScoped<GerenciadorArquivo>();
17 builder.Services.AddScoped<EmprestimoLivroAspCore.Cookie.Cookie>();
18 builder.Services.AddScoped<EmprestimoLivroAspCore.CarrinhoCompra.CookieCarrinhoCompra>();
19
20 var app = builder.Build();
```

